

Array

Array

Come usare gli array in C++ per memorizzare sequenze di valori dello stesso tipo.

Cos'è un array?

Immagina di dover memorizzare i voti di 30 studenti. Creeresti 30 variabili diverse? `voto1`, `voto2`, ... `voto30`? Sarebbe un disastro.

Un **array** è una fila di scatole numerate che contengono valori dello stesso tipo. Un solo nome, tanti valori. Puoi accedere a ciascuna scatola usando il suo numero (chiamato **indice**).

Dichiarare un array

```
tipo nome[dimensione];
```

```
int numeri[5];      // una fila di 5 numeri interi
double voti[10];    // una fila di 10 numeri decimali
char lettere[26];    // una fila di 26 caratteri
```

La dimensione deve essere un numero fisso, noto in anticipo (prima dell'esecuzione del programma).

Inizializzare un array

Puoi assegnare i valori subito alla dichiarazione, tra parentesi graffe:

```
int numeri[5] = {10, 20, 30, 40, 50};
double voti[4] = {8.5, 7.0, 9.0, 6.5};
char vocali[5] = {'a', 'e', 'i', 'o', 'u'};
```

Se non scrivi la dimensione, viene calcolata automaticamente dai valori:

```
int numeri[] = {10, 20, 30, 40, 50}; // dimensione 5, calcolata
automaticamente
```

Per inizializzare tutti gli elementi a zero:

```
int numeri[5] = {0}; // tutti e 5 a zero
int numeri2[5] = {}; // anche questo funziona in C++11
```

Se fornisci meno valori della dimensione, il resto viene azzerato:

```
int numeri[5] = {1, 2}; // diventa {1, 2, 0, 0, 0}
```

Accedere agli elementi

Ogni elemento ha un numero d'ordine chiamato **indice**. Attenzione: gli indici partono da **0**, non da 1.

```
int numeri[] = {10, 20, 30, 40, 50};
//indice:      0   1   2   3   4

cout << numeri[0] << endl; // 10 - primo elemento
cout << numeri[1] << endl; // 20
cout << numeri[4] << endl; // 50 - ultimo elemento
```

Per un array di dimensione **n**, gli indici vanno da **0** a **n-1**.

Modificare un elemento

```
int numeri[] = {10, 20, 30};
numeri[1] = 99;           // cambiamo il secondo elemento
cout << numeri[1] << endl; // 99
```

Pericolo: uscire dai limiti

In C++ non c'è alcun controllo automatico sui limiti. Se accedi a un indice che non esiste, il comportamento è imprevedibile e può corrompere la memoria del programma:

```
int numeri[5] = {1, 2, 3, 4, 5};
cout << numeri[10] << endl; // PERICOLO: indice fuori limiti
numeri[10] = 99;           // PERICOLO: potrebbe causare crash o dati
corrotti
```

Fai sempre attenzione che gli indici siano compresi tra `0` e `dimensione - 1`.

Scorrere un array con `for`

Il modo più comune per lavorare con tutti gli elementi di un array:

```
int numeri[] = {10, 20, 30, 40, 50};
int n = 5;

for (int i = 0; i < n; i++) {
    cout << numeri[i] << " ";
}
// Output: 10 20 30 40 50
```

Calcolare la dimensione automaticamente

Puoi ricavare il numero di elementi con `sizeof` :

```
int numeri[] = {10, 20, 30, 40, 50};  
// sizeof(numeri) = dimensione totale in byte dell'array  
// sizeof(numeri[0]) = dimensione in byte di un singolo elemento  
int dimensione = sizeof(numeri) / sizeof(numeri[0]);  
cout << dimensione << endl; // 5
```

Scorrere senza indice (range-based for)

In C++ moderno puoi scorrere tutti gli elementi in modo più semplice:

```
int numeri[] = {10, 20, 30, 40, 50};  
  
for (int n : numeri) {  
    cout << n << " ";  
}  
// Output: 10 20 30 40 50
```

Si legge: "per ogni `n` nell'array `numeri`".

Operazioni comuni

Trovare il valore massimo

```
int numeri[] = {3, 7, 1, 9, 4};
int n = 5;
int massimo = numeri[0]; // partiamo dal primo elemento

for (int i = 1; i < n; i++) {
    if (numeri[i] > massimo) {
        massimo = numeri[i]; // aggiorniamo il massimo se troviamo un
        valore più grande
    }
}

cout << "Massimo: " << massimo << endl; // 9
```

Calcolare somma e media

```
int numeri[] = {5, 10, 15, 20, 25};
int n = 5;
int somma = 0;

for (int i = 0; i < n; i++) {
    somma += numeri[i];
}

double media = (double)somma / n;
cout << "Somma: " << somma << endl; // 75
cout << "Media: " << media << endl; // 15
```

Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    const int N = 5;
    int voti[N];

    // Raccogliamo i voti dall'utente
    cout << "Inserisci " << N << " voti:" << endl;
    for (int i = 0; i < N; i++) {
        cout << "Voto " << (i + 1) << ": ";
        cin >> voti[i];
    }

    // Calcoliamo somma, massimo e minimo
    int somma = 0;
    int max = voti[0];
    int min = voti[0];

    for (int i = 0; i < N; i++) {
        somma += voti[i];
        if (voti[i] > max) max = voti[i];
        if (voti[i] < min) min = voti[i];
    }

    double media = (double)somma / N;

    cout << endl;
    cout << "Media: " << media << endl;
    cout << "Voto più alto: " << max << endl;
    cout << "Voto più basso: " << min << endl;

    return 0;
}
```